

MLOps Assignment 01: End-to-End ML Model Development, CI/CD, and Production Deployment

****Student Name:**** [Your Name]

****Student ID:**** [Your ID]

****Course:**** MLOps - Experimental Learning

****Date:**** June 2026

Table of Contents

1. Introduction
2. Problem Statement
3. Dataset Overview
4. Setup and Installation
5. Exploratory Data Analysis (EDA)
6. Feature Engineering
7. Model Development
8. Experiment Tracking with MLflow
9. Model Packaging and Reproducibility
10. CI/CD Pipeline
11. Model Containerization
12. Production Deployment
13. Monitoring and Logging
14. Architecture and Workflow
15. Challenges Faced
16. Lessons Learned
17. Conclusion
18. References

1. Introduction

This project was part of my MLOps coursework where I had to build an end-to-end machine learning pipeline. The goal was to go beyond just training a model and actually deploy it like a real production system. I learned that there's a lot more to MLOps than just the ML part - there's infrastructure, automation, monitoring, and all the operational aspects that make a model actually useful in production.

Honestly, I underestimated how complex this would be. Initially I thought it would be just training a model and maybe putting it in a Docker container, but it turned out to be a whole system with many moving parts. This report documents my journey through building this pipeline, the decisions I made, challenges I faced, and what I learned along the way.

2. Problem Statement

The task was to build a machine learning classifier to predict heart disease risk based on patient health data. The dataset comes from the UCI Machine Learning Repository and contains various clinical attributes like age, blood pressure, cholesterol levels, etc. The target is binary - either the patient has heart disease or not.

What made this interesting from an MLOps perspective was that I had to deploy this as a cloud-ready, monitored API. This meant I couldn't just train a model and be done with it - I had to think about how it would actually be used in production, how to monitor it, how to ensure it works reliably, and how to automate the whole process.

3. Dataset Overview

****Dataset:**** Heart Disease UCI Dataset
****Source:**** <https://archive.ics.uci.edu/ml/datasets/Heart+Disease>
****Samples:**** 303 patients
****Features:**** 14 clinical attributes
****Target:**** Binary (0 = No heart disease, 1 = Heart disease)

Features:

- ****age****: Patient age in years
- ****sex****: Gender (1 = male, 0 = female)
- ****cp****: Chest pain type (1-4)
- ****trestbps****: Resting blood pressure (mm Hg)
- ****chol****: Serum cholesterol (mg/dl)
- ****fbs****: Fasting blood sugar > 120 mg/dl (1 = true, 0 = false)
- ****restecg****: Resting electrocardiographic results (0-2)
- ****thalach****: Maximum heart rate achieved
- ****exang****: Exercise induced angina (1 = yes, 0 = no)
- ****oldpeak****: ST depression induced by exercise
- ****slope****: Slope of peak exercise ST segment
- ****ca****: Number of major vessels colored by fluoroscopy
- ****thal****: Thalassemia (3 = normal, 6 = fixed defect, 7 = reversible defect)

I initially had some trouble downloading the dataset - the main UCI URL wasn't working reliably, so I ended up using the `processed.cleveland.data` file which worked much better. The dataset had some missing values represented as '?' which I had to handle during preprocessing.

4. Setup and Installation

Prerequisites

- Python 3.10+
- Docker
- Kubernetes (Minikube or Docker Desktop)
- Git

Installation Steps

```

```bash
1. Clone the repository
git clone <your-repo-url>
cd MLOPS_ASSIGNMENT

2. Create virtual environment
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate

3. Install dependencies
pip install -r requirements.txt
```

```

I initially tried Python 3.11 but ran into some compatibility issues with MLflow, so I downgraded to Python 3.10 which was more stable. This was one of those things I learned the hard way - always check package compatibility before committing to a Python version.

```

### Key Dependencies
- **scikit-learn**: For ML models
- **fastapi**: For the API
- **mlflow**: For experiment tracking
- **docker**: For containerization
- **pytest**: For testing
- **prometheus-client**: For monitoring

```

5. Exploratory Data Analysis (EDA)

Initial Data Exploration

I started by loading the data and checking its basic structure:

```

```python
df = pd.read_csv('data/heart.csv')
print(df.shape) # (303, 14)
print(df.info())
```

```

The dataset had 303 samples with 14 features, which was a reasonable size for this project. I was relieved that it wasn't too large - training would be quick.

Class Distribution

One of the first things I checked was the class balance:

![Class Distribution](screenshots/class_distribution.png)

The dataset was reasonably balanced - about 54% had heart disease and 46% didn't. This was good because I wouldn't need to do any oversampling or undersampling, which can get complicated.

Feature Distributions

I looked at the distributions of numerical features:

![Numerical Distributions](screenshots/numerical_distributions.png)

Some observations:

- Age was normally distributed around 50-60 years
- Cholesterol had some outliers that might need attention
- Oldpeak was right-skewed - most values were near 0

This helped me understand that I'd need to handle outliers, especially in cholesterol.

Correlation Analysis

I created a correlation heatmap to understand feature relationships:

![Correlation Heatmap](screenshots/correlation_heatmap.png)

Key findings:

- **Positive correlation with target**: cp (chest pain type), thalach (max heart rate), slope
- **Negative correlation with target**: oldpeak, ca, thal, exang

This was really helpful because it told me which features were likely to be important for the model. I made sure to pay special attention to these during feature engineering.

Categorical Feature Analysis

I also looked at how categorical features related to the target:

![Categorical Analysis](screenshots/categorical_analysis.png)

Chest pain type (cp) was surprisingly predictive - different types of chest pain had very different heart disease rates. Exercise induced angina (exang) also showed clear differences.

EDA Takeaways

From the EDA, I learned:

1. The dataset is clean enough after handling missing values
2. Class balance is good - no special handling needed
3. Some features (cp, thalach, oldpeak, ca, thal) are strongly correlated with target
4. There are some outliers in cholesterol that might need handling
5. The dataset is small enough that complex models might overfit

These insights guided my modeling decisions later.

6. Feature Engineering

Data Cleaning

The UCI dataset had missing values represented as '?' which I had to handle:

```
```python
df = pd.read_csv('data/raw_heart.data', names=column_names, na_values="?")
```
```

I used median imputation for missing values because the data had some outliers, and median is more robust than mean in such cases. I initially tried mean imputation but median gave better results.

Feature Encoding

The categorical features (cp, restecg, slope, thal) needed to be encoded. I used one-hot encoding:

```
```python
categorical_cols = ["cp", "restecg", "slope", "thal"]
df = pd.get_dummies(df, columns=categorical_cols, drop_first=True)
```
```

I used `drop\_first=True` to avoid multicollinearity. This increased the dimensionality but worked well for tree-based models. For logistic regression, I might have tried label encoding instead, but one-hot seemed to work fine.

Feature Scaling

I used StandardScaler for numerical features:

```
```python
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```
```

StandardScaler assumes Gaussian distribution, which might not be ideal for all features. I considered using MinMaxScaler for non-normal distributions, but StandardScaler worked well enough for this dataset.

Saving Preprocessing Artifacts

One of the tricky parts was ensuring reproducibility during inference. I had to save:

- The fitted scaler
- The feature column names (for alignment during prediction)

```
```python
with open("models/scaler.pkl", "wb") as f:
 pickle.dump(scaler, f)

with open("models/feature_columns.pkl", "wb") as f:
 pickle.dump(feature_cols, f)
```
```

...

This was crucial because during prediction, the input needs to have the exact same columns as training, in the same order.

7. Model Development

Model Selection

I chose to implement two models:

1. **Logistic Regression**: Simple, interpretable, good baseline
2. **Random Forest**: Handles non-linear relationships, often performs better

I initially considered trying SVM and XGBoost as well, but they took longer to train and I wanted to focus on getting the pipeline right rather than model complexity. LR and RF gave me a good baseline to work with.

Logistic Regression

```
```python
params = {
 "C": 1.0,
 "max_iter": 200,
 "solver": "lbfgs",
 "random_state": 42
}
model = LogisticRegression(**params)
```
```

I tried different C values but 1.0 seemed to work best. I could have experimented with different solvers (liblinear, saga) for comparison, but lbfgs worked fine.

Random Forest

```
```python
params = {
 "n_estimators": 100,
 "max_depth": 6,
 "min_samples_split": 5,
 "random_state": 42
}
model = RandomForestClassifier(**params)
```
```

I used conservative values to prevent overfitting. The max_depth of 6 and min_samples_split of 5 were starting points - I could tune these further with hyperparameter optimization, but for this assignment, they worked well enough.

Cross-Validation

I used 5-fold cross-validation to check for overfitting:

```
```python
cv_scores = cross_val_score(model, X_train, y_train, cv=5, scoring="accuracy")
cv_mean = round(np.mean(cv_scores), 4)
cv_std = round(np.std(cv_scores), 4)
```
```

This was important because the dataset is small and I wanted to ensure the model wasn't just memorizing the training data.

Evaluation Metrics

I evaluated using multiple metrics:

- **Accuracy**: Overall correctness
- **Precision**: Of predicted positive, how many are actually positive
- **Recall**: Of actual positives, how many were predicted positive
- **ROC-AUC**: Threshold-independent performance measure

I focused on ROC-AUC as the primary metric for model selection since it's threshold-independent and gives a good overall picture of model performance.

Model Performance

Based on my training runs:

- **Logistic Regression**: ROC-AUC ~0.85-0.90
- **Random Forest**: ROC-AUC ~0.88-0.92

Random Forest generally performed better, likely due to non-linear relationships in the data. I selected the best model automatically based on ROC-AUC score.

8. Experiment Tracking with MLflow

MLflow Setup

I used MLflow for tracking all my experiments:

```
```python
exp_name = "Heart_Disease_Classifier"
mlflow.set_experiment(exp_name)
```
```

Initially, I had some issues with MLflow not showing experiments in the UI. It turned out I needed to run `mlflow ui` from the project root directory - working directory matters for MLflow tracking.

What I Tracked

For each model, I logged:

- **Parameters**: Model hyperparameters
- **Metrics**: Accuracy, precision, recall, F1, ROC-AUC
- **Artifacts**: Confusion matrix plots, feature importance plots
- **CV metrics**: Mean and std of cross-validation scores

MLflow UI

![MLflow UI](screenshots/mlflow_ui.png)

The MLflow UI was really helpful for comparing different model configurations. I could see which parameters worked best and how different models compared.

Experiment Runs

I created separate runs for each model:

- `LogisticRegression\_Run`
- `RandomForest\_Run`

Each run had all the relevant metrics and artifacts logged. This made it easy to go back and see exactly what I did for each experiment.

9. Model Packaging and Reproducibility

Model Saving

I saved the best model using pickle:

```
``python
with open("models/best_model.pkl", "wb") as f:
    pickle.dump(best_model, f)
``
```

I initially considered using MLflow's model registry, but pickle was simpler for this assignment and worked well enough.

Requirements File

I created a clean `requirements.txt` with all dependencies:

...

```
# Core ML
scikit-learn==1.3.0
pandas==2.0.3
numpy==1.24.3
```

```
# Experiment Tracking
mlflow==2.7.1
```



```
# API
fastapi==0.103.1
uvicorn[standard]==0.23.2
pydantic==2.3.0

# Monitoring
prometheus-client==0.19.0

# Testing
pytest==7.4.2

# Visualization
matplotlib==3.7.2
seaborn==0.12.2
``
```

I pinned specific versions to ensure reproducibility. This was important because different versions can have different behaviors.

Preprocessing Pipeline

For full reproducibility, I saved the entire preprocessing pipeline:

- The fitted scaler
- Feature column names
- Encoding logic

This ensured that during inference, the exact same transformations would be applied as during training.

Reproducibility Testing

I tested reproducibility by:

1. Training the model in a fresh environment
2. Loading the saved artifacts
3. Running predictions on the same test data
4. Verifying results matched

This caught some issues with feature alignment that I had to fix.

10. CI/CD Pipeline

GitHub Actions Workflow

I created a CI/CD pipeline using GitHub Actions:

```
``yaml
name: CI/CD Pipeline
```

on: [push, pull_request]

jobs:

 linting:

 runs-on: ubuntu-latest

 steps:

 - uses: actions/checkout@v3

 - name: Set up Python

 uses: actions/setup-python@v4

 with:

 python-version: '3.10'

 - name: Install dependencies

 run: pip install -r requirements.txt

 - name: Lint with flake8

 run: flake8 src/ tests/ --count --select=E9,F63,F7,F82 --show-source --statistics

 test:

 runs-on: ubuntu-latest

 steps:

 - uses: actions/checkout@v3

 - name: Set up Python

 uses: actions/setup-python@v4

 with:

 python-version: '3.10'

 - name: Install dependencies

 run: pip install -r requirements.txt

 - name: Run tests

 run: pytest tests/test_pipeline.py -v

 train:

 runs-on: ubuntu-latest

 steps:

 - uses: actions/checkout@v3

 - name: Set up Python

 uses: actions/setup-python@v4

 with:

 python-version: '3.10'

 - name: Install dependencies

 run: pip install -r requirements.txt

 - name: Train models

 run: python src/train.py

 docker:

 runs-on: ubuntu-latest

 steps:

 - uses: actions/checkout@v3

 - name: Set up Docker Buildx

 uses: docker/setup-buildx-action@v2

 - name: Build Docker image

 run: docker build -t heart-disease-api .

```
- name: Test Docker image
  run: docker run heart-disease-api python -c "import app"
...
```

This was my first time setting up a proper CI/CD pipeline, so it was a learning experience. The pipeline:

1. Runs linting to check code quality
2. Executes unit tests
3. Trains the model
4. Builds and tests the Docker image

CI/CD Pipeline Run

![CI/CD Pipeline](screenshots/ci_cd_pipeline.png)

The pipeline runs automatically on every push and pull request. If any step fails, the whole pipeline fails and I get clear error messages.

Issues Faced

Initially, the pipeline was failing due to missing dependencies in requirements.txt. I had to add all the development dependencies explicitly. I also had to create the models directory before the training step, otherwise the training would fail when trying to save the model.

11. Model Containerization

Dockerfile

I created a Dockerfile to containerize the application:

```
``dockerfile
FROM python:3.10-slim

WORKDIR /heart-mlops

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

RUN mkdir -p logs

EXPOSE 8000

CMD ["python", "app/app.py"]
...
```

I initially tried a multi-stage build to reduce image size, but I had issues with model files not being copied correctly. I switched to a single-stage build which was simpler and the image size wasn't too large anyway.

Building the Image

```
```bash
docker build -t heart-disease-api .
```
```

Testing Locally

```
```bash
docker run -p 8000:8000 heart-disease-api
```
```

I tested the container locally by:

1. Building the image
2. Running the container
3. Making sample API calls
4. Verifying predictions worked

This caught some issues with file paths that were different inside the container.

API Endpoints

The container exposes:

- `GET /` - API status
- `GET /health` - Health check
- `GET /metrics` - Prometheus metrics
- `POST /predict` - Heart disease prediction

Sample API Call

```
```bash
curl -X POST http://localhost:8000/predict \
 -H "Content-Type: application/json" \
 -d
'{"age":54,"sex":1,"cp":2,"trestbps":130,"chol":250,"fbs":0,"restecg":1,"thalach":150,"exang":0,"oldpeak":1.
5,"slope":2,"ca":0,"thal":2}'
```
```

Response:

```
```json
{
 "status": "success",
 "result": {
 "prediction": 1,
 "label": "Heart Disease Detected",
 "confidence": 0.8234
 },
 "timestamp": "2026-06-29T12:00:00.000000"
}
```

...

---

## ## 12. Production Deployment

### ### Kubernetes Deployment

I deployed the containerized API to Kubernetes using Minikube for local testing.

#### ### Deployment Manifest

```
```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: heart-disease-api
spec:
  replicas: 2
  selector:
    matchLabels:
      app: heart-disease-api
  template:
    metadata:
      labels:
        app: heart-disease-api
    spec:
      containers:
        - name: heart-disease-api
          image: heart-disease-api:latest
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8000
      resources:
        requests:
          memory: "256Mi"
          cpu: "250m"
        limits:
          memory: "512Mi"
          cpu: "500m"
      livenessProbe:
        httpGet:
          path: /health
          port: 8000
        initialDelaySeconds: 10
        periodSeconds: 15
      readinessProbe:
        httpGet:
          path: /health
          port: 8000
```

```
        initialDelaySeconds: 5
        periodSeconds: 10
...

```

I used 2 replicas for high availability. The resource limits were conservative - I can adjust them based on actual usage.

Service Manifest

```
```yaml
apiVersion: v1
kind: Service
metadata:
 name: heart-disease-service
spec:
 selector:
 app: heart-disease-api
 type: LoadBalancer
 ports:
 - protocol: TCP
 port: 80
 targetPort: 8000
...

```

I initially used NodePort for local testing, then switched to LoadBalancer for cloud deployment. For local testing with Minikube, NodePort works better.

### ### Deployment Steps

```
```bash
# Build and load Docker image into Minikube
eval $(minikube docker-env)
docker build -t heart-disease-api .

# Apply deployment
kubectl apply -f k8s/deployment.yaml

# Check status
kubectl get pods
kubectl get services
...

```

Deployment Verification

![Kubernetes Pods](screenshots/k8s_pods.png)

![Kubernetes Services](screenshots/k8s_services.png)

Health Checks

I added liveness and readiness probes:

- ****Liveness probe****: Checks if the container is still running
- ****Readiness probe****: Checks if the container is ready to receive traffic

Initially, the pods kept restarting due to health check failures. I had to increase the `initialDelaySeconds` to give the app enough time to start up.

Accessing the Service

For Minikube:

```
``bash
minikube service heart-disease-service
``
```

This opens the service in the browser.

13. Monitoring and Logging

API Logging

I added logging to the FastAPI application:

```
``python
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s",
    handlers=[
        logging.FileHandler("logs/api_requests.log"),
        logging.StreamHandler()
    ]
)
``
```

Logs are written to both file and console, showing:

- Request timestamps
- Input data
- Prediction results
- Error messages

Prometheus Metrics

I integrated Prometheus metrics:

```
``python
from prometheus_client import Counter, Histogram, generate_latest
```

```
PREDICTION_REQUESTS = Counter(
    'prediction_requests_total',
```

```

    'Total prediction requests',
    ['status']
)

PREDICTION_LATENCY = Histogram(
    'prediction_latency_seconds',
    'Prediction request latency'
)

HEART_DISEASE_PREDICTIONS = Counter(
    'heart_disease_predictions_total',
    'Total heart disease predictions',
    ['prediction']
)
...

```

Metrics available:

- `prediction\_requests\_total` - Total requests (by status)
- `prediction\_latency\_seconds` - Request latency
- `heart\_disease\_predictions\_total` - Predictions (by class)

Metrics Endpoint

The `/metrics` endpoint exposes Prometheus metrics:

```

```bash
curl http://localhost:8000/metrics
```

```

This can be scraped by Prometheus for monitoring.

Monitoring Dashboard

While I didn't set up a full Grafana dashboard (that would be a future improvement), the metrics endpoint provides the foundation for monitoring. In a real production environment, I would add:

- Grafana for visualization
- Alerting for anomalous behavior
- Log aggregation with ELK stack

14. Architecture and Workflow

System Architecture

The overall architecture consists of:

1. **Data Layer**: UCI dataset download script
2. **Training Pipeline**: Preprocessing, model training, MLflow tracking
3. **Inference Layer**: Model loading, prediction logic

4. ****API Layer****: FastAPI application serving predictions
5. ****Containerization****: Docker container
6. ****Orchestration****: Kubernetes deployment
7. ****CI/CD****: GitHub Actions pipeline
8. ****Monitoring****: Prometheus metrics and logging

Data Flow

1. Data is downloaded from UCI repository
2. Data is cleaned, encoded, and scaled
3. Models are trained with MLflow tracking
4. Best model is selected and saved
5. API loads model and serves predictions
6. Docker container packages the application
7. Kubernetes deploys and scales the application
8. Monitoring tracks performance and usage

CI/CD Workflow

![CI/CD Workflow](screenshots/ci_cd_workflow.png)

The CI/CD pipeline:

1. Triggers on push/pull request
2. Lints code for quality
3. Runs unit tests
4. Trains the model
5. Builds Docker image
6. Tests Docker image
7. Deploys to Kubernetes (manual step)

15. Challenges Faced

Data Pipeline Challenges

****Issue****: UCI dataset URL kept failing

****Solution****: Found the processed.cleveland.data file which worked reliably

****Lesson****: Always have fallback URLs for external data sources

****Issue****: Missing values represented as '?' not NaN

****Solution****: Added `na\_values="?"` parameter when reading CSV

****Lesson****: Always check how missing values are represented in raw data

****Issue****: Feature alignment during inference

****Solution****: Saved feature columns during training, used `reindex()` during inference

****Lesson****: Always save and load exact feature order from training

Model Training Challenges

****Issue****: MLflow UI not showing experiments
****Solution****: Needed to run mlflow ui from project root directory
****Lesson****: Working directory matters for MLflow tracking

****Issue****: CV scores very different from test scores
****Solution****: Added stratified splitting to maintain class balance
****Lesson****: Stratification is important for imbalanced datasets

API Development Challenges

****Issue****: Module import path problems
****Solution****: Added sys.path manipulation to include src directory
****Lesson****: Python path issues are common in multi-directory projects

****Issue****: Uvicorn reload causing issues
****Solution****: Set reload=False in uvicorn.run()
****Lesson****: Reload can be problematic with complex imports

Containerization Challenges

****Issue****: Model files not in Docker container
****Solution****: Added model training step in CI/CD before Docker build
****Lesson****: Container needs all artifacts at build time

Kubernetes Deployment Challenges

****Issue****: Pods kept restarting due to health check failures
****Solution****: Increased initialDelaySeconds in liveness probe
****Lesson****: Services need time to start before health checks

****Issue****: LoadBalancer service not getting external IP
****Solution****: Used NodePort instead for local testing
****Lesson****: LoadBalancer requires cloud provider or specific setup

CI/CD Pipeline Challenges

****Issue****: Pipeline failing on missing dependencies
****Solution****: Added all dependencies including dev packages to requirements.txt
****Lesson****: CI environment needs all dependencies explicitly listed

16. Lessons Learned

Technical Skills

****MLOps Pipeline****: Learned how to build end-to-end ML pipelines

- Data acquisition and preprocessing
- Model training and evaluation
- Experiment tracking

- Model packaging
- API development
- Containerization
- Orchestration
- Monitoring

****Experiment Tracking****: MLflow for tracking experiments

- Logging parameters and metrics
- Saving artifacts
- Comparing experiments
- Model versioning

****Containerization****: Docker for reproducible deployments

- Writing Dockerfiles
- Building images
- Running containers
- Debugging container issues

****Orchestration****: Kubernetes for scaling deployments

- Writing deployment manifests
- Services and networking
- Health checks
- Resource management

****CI/CD****: GitHub Actions for automation

- Writing workflows
- Linting and testing
- Automated builds
- Artifact management

****Monitoring****: Prometheus for application metrics

- Defining metrics
- Exposing metrics endpoint
- Basic monitoring setup

Soft Skills

****Debugging****: Spent a lot of time debugging integration issues

- Feature alignment problems
- Path issues in containers
- Health check timing
- Dependency conflicts

****Documentation****: Learned importance of documenting challenges

- Writing this report
- Documenting architecture
- Keeping notes on decisions

****Patience****: MLOps involves many moving parts

- Things take time to get right

- Iterative development is necessary
- Testing at each stage is crucial

****Iterative Development****: Built incrementally

- Started with basic model
- Added monitoring
- Added CI/CD
- Added deployment
- Each step built on the previous

Things I'd Do Differently Next Time

1. ****Start with simpler model****: Before adding complexity
2. ****Set up monitoring earlier****: In the development process
3. ****Add more comprehensive error handling****: Throughout the codebase
4. ****Write integration tests earlier****: Not just unit tests
5. ****Document architecture decisions****: As I make them, not at the end
6. ****Add hyperparameter tuning****: With Optuna or GridSearch
7. ****Implement model versioning****: For A/B testing
8. ****Add input validation****: For API endpoints
9. ****Set up Grafana dashboard****: For better monitoring
10. ****Add model drift detection****: For production monitoring

17. Conclusion

This assignment was a comprehensive learning experience in MLOps. I went from just training a model to building a complete production-ready system. Here's what I accomplished:

****Technical Achievements:****

- Built end-to-end ML pipeline
- Integrated experiment tracking with MLflow
- Created CI/CD pipeline with GitHub Actions
- Containerized application with Docker
- Deployed to Kubernetes
- Added monitoring with Prometheus

****Learning Outcomes:****

- Deep understanding of MLOps practices
- Experience with modern MLOps tools
- Ability to build production ML systems
- Appreciation for operational complexity

****Challenges Overcome:****

- Data pipeline issues
- Feature alignment problems
- Containerization challenges
- Kubernetes deployment complexities
- CI/CD pipeline debugging

The project is now ready for submission with all required components:

-  Data download script
-  EDA with visualizations
-  Two models with cross-validation
-  MLflow experiment tracking
-  Unit tests
-  CI/CD pipeline
-  Docker container
-  Kubernetes deployment
-  Monitoring
-  This report

While there's always room for improvement (hyperparameter tuning, better monitoring, etc.), the project demonstrates a solid understanding of MLOps principles and practices. The experience of building this from scratch has given me confidence that I can apply these skills to real-world production ML systems.

18. References

1. **UCI Machine Learning Repository** - Heart Disease Dataset
<https://archive.ics.uci.edu/ml/datasets/Heart+Disease>
2. **MLflow Documentation**
<https://mlflow.org/docs/latest/index.html>
3. **FastAPI Documentation**
<https://fastapi.tiangolo.com/>
4. **Docker Documentation**
<https://docs.docker.com/>
5. **Kubernetes Documentation**
<https://kubernetes.io/docs/>
6. **GitHub Actions Documentation**
<https://docs.github.com/en/actions>
7. **Prometheus Documentation**
<https://prometheus.io/docs/>
8. **Scikit-learn Documentation**
<https://scikit-learn.org/stable/>
9. **Pytest Documentation**
<https://docs.pytest.org/>

Appendix A: Code Repository

The complete code is available at:
[GitHub Repository URL]

To run the project:

```
```bash
git clone <repository-url>
cd MLOPS_ASSIGNMENT
pip install -r requirements.txt
python data/download_data.py
python src/train.py
python app/app.py
```
```

Appendix B: Screenshots

[Screenshots folder with images of:

- MLflow UI
- CI/CD pipeline runs
- Kubernetes deployment
- API testing
- EDA visualizations
- Model performance metrics]

****End of Report****